COSC264 · INTRODUCTION TO COMPUTER NETWORKS AND THE INTERNET · TERM 4 · MODULE 6

Application Protocols: the Web, Email and DNS

1 · Network applications | 2 · The Web and HTTP | 3 · Email | 4 · DNS

cosc264.up.railway.app

OUTLINE

Module 6 — roadmap

- General concepts about network applications
- One network application: the Web
- HTTP: the protocol that supports the Web
- Email
- DNS: the Domain Name System

MODULE 6

Network Applications

What an application needs from the layers beneath it

1 · Network applications | 2 · The Web and HTTP | 3 · Email | 4 · DNS

NETWORK APPLICATIONS

What is a network application?

Programs that run on different end systems and communicate with each other over the network.

Web browsing

Google Chrome, Mozilla Firefox, Microsoft Edge

Email

Gmail, Outlook

Instant messaging

Slack, WhatsApp, Facebook Messenger

Streaming services

YouTube, Netflix

Social media

Instagram, TikTok

AI assistants

Claude, ChatGPT, Gemini

NETWORK APPLICATIONS

Application-layer protocols

An **application-layer protocol** is one piece of a network application — but a big piece.

It defines how the application's processes, running on different end systems, **pass messages to each other**: which messages exist, their format, and when and how to respond.

This is what lets programs on different machines work together — e.g. any web browser can fetch a page from any web server, whoever wrote each program.

Network application	Application-layer protocol
Web application	HTTP
Email	SMTP

Choosing a transport protocol

- Application-layer protocols are controlled by the **application developer**; transport-layer protocols, however, are built into the **OS**.
- As a developer you can only choose between the transport protocols the OS provides — so pick the one that best fits what your application-layer protocol needs.

Property	Example applications	Transport protocol
Reliable data transfer	Email, IM, file transfer, financial apps	**TCP** preferred UDP unsuitable
Low delay	Internet telephony, teleconferencing, multiplayer games	**UDP** preferred TCP retransmissions add latency

MODULE 6

The Web and HTTP

The application-layer protocol behind every page you load

1 · Network applications | **2 · The Web and HTTP** | 3 · Email | 4 · DNS

THE WEB

The World Wide Web

THE WEB

The **World Wide Web** (aka Web) is a **network application**: a global collection of documents and other resources shared over the Internet — like a large file storage system.

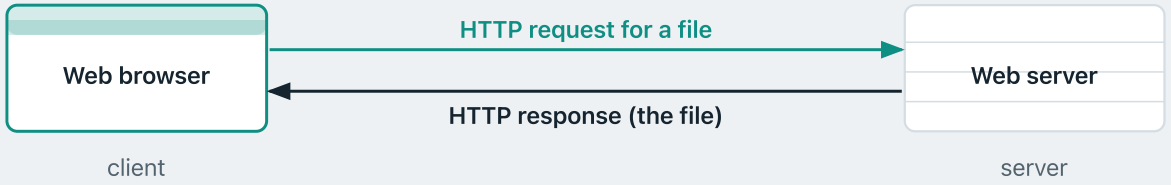
Each object on the Web (e.g., an image, an HTML file) is addressable by a **URL**:

```
http://www.someschool.edu/someDept/pic.gif
     └── host name ──┘ └──── path name ─────┘
```

HTTP: the protocol behind the Web

HTTP The **Hypertext Transfer Protocol** is the **application-layer protocol** that supports the Web — it defines how clients request objects by URL and how servers respond.

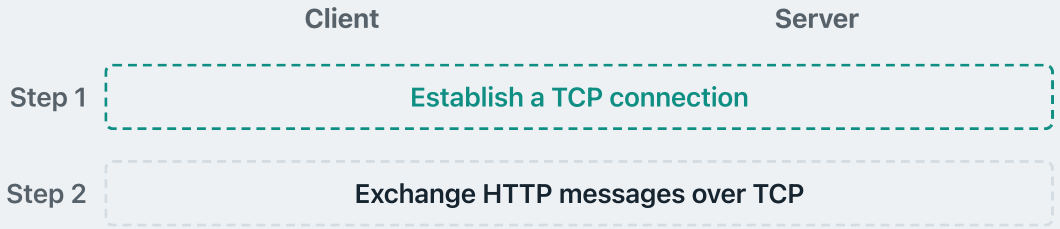
HTTP follows a **request–response** pattern: the client sends an **HTTP request** for an object, and the server answers with an **HTTP response** carrying it.



HTTP

How HTTP works

HTTP uses **TCP** as its transport protocol — so before any HTTP message is sent, a TCP connection must be established first.



STATELESS HTTP is **stateless** — the server maintains no information about past client requests. Each request is handled independently.

HTTP connections

Non-persistent HTTP

- At most **one object** is sent over a TCP connection — connection is then closed
- HTTP/1.0 uses non-persistent HTTP

Persistent HTTP

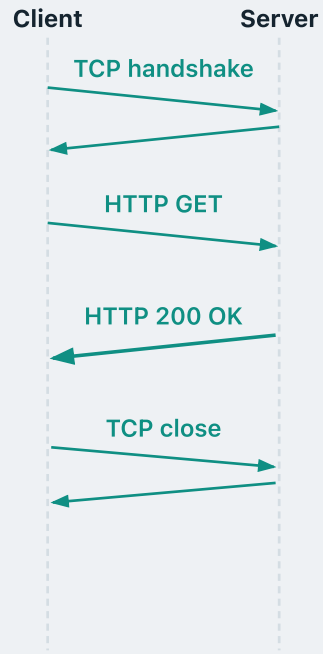
- **Multiple objects** can be sent over a single TCP connection
- HTTP/1.1, HTTP/2, and HTTP/3 use persistent HTTP

Protocol	Published	Connection type	Status
HTTP/1.0	1996	Non-persistent	Legacy
HTTP/1.1	1997	Persistent	Standard
HTTP/2	2015	Persistent	Standard
HTTP/3	2022	Persistent (QUIC)	Standard

Non-persistent HTTP in action

Suppose a client wants to retrieve a file from: `http://www.someSchool.edu/someDepartment/home.index`

1. Client establishes a **TCP connection** with the server
2. Client sends an **HTTP GET** request — path: `/someDepartment/home.index`
3. Server retrieves the HTML file, wraps it in an **HTTP response**, and sends it back
4. Server signals TCP to **close the connection**



Response time: non-persistent HTTP

RESPONSE TIME — The time to retrieve a single object using HTTP — from the client's request until the object is received

Q: What is the response time for non-persistent HTTP?

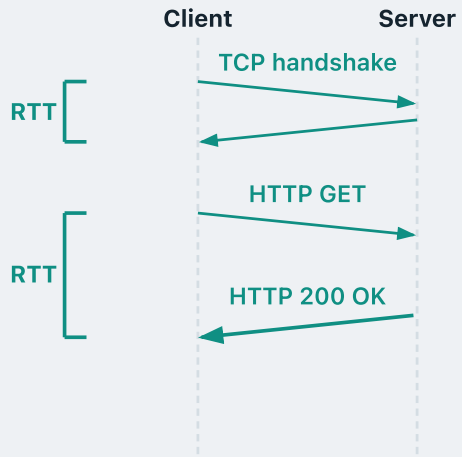
A: 2 RTTs + file transmission time

RTT — Recall: **round-trip time** — the time for a small packet to travel from client to server and back

First RTT: the TCP handshake — client sends SYN, server replies SYN-ACK

Second RTT: client sends HTTP GET, server sends back the response

With many referenced objects, this cost multiplies — a significant burden on the server



Persistent HTTP in action

Client and server establish a TCP connection and exchange HTTP messages

Server **keeps the TCP connection open** after sending the first HTTP response

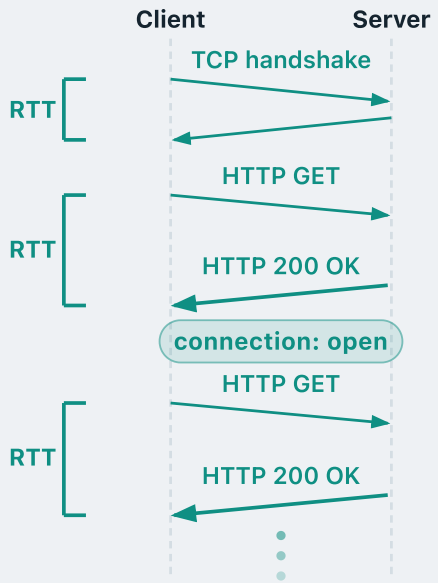
Subsequent HTTP messages between the same client and server use the **same TCP connection** — no new handshake needed

Response time on one persistent connection:

1 RTT for the initial TCP handshake, then 1 RTT for each HTTP request.

So for n requests: **$\text{total} \approx (1 + n)$ RTT** (plus transmission time).

Amortised over the n objects: $(1 + n)/n = (1/n + 1)$ RTT. As n becomes large, $1/n \to 0$, so this approaches **~1 RTT per object**.

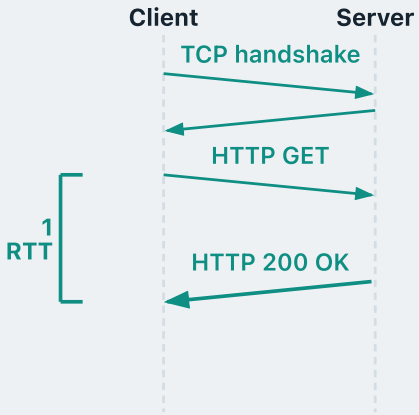


Can we do even better than 1 RTT per object?

Improving HTTP efficiency further with pipelining

WITHOUT PIPELINING

Client issues a new request **only after** receiving the previous response — **1 RTT per file**



Improving HTTP efficiency further with pipelining

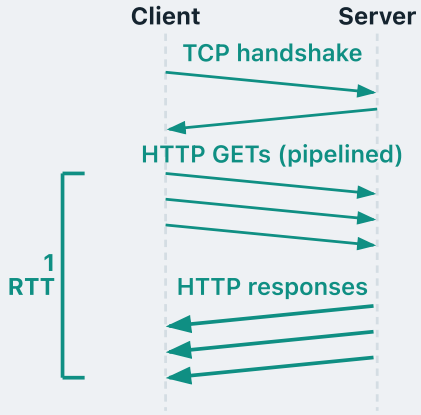
WITHOUT PIPELINING

Client issues a new request **only after** receiving the previous response — **1 RTT per file**

WITH PIPELINING

Client sends requests **as soon as it needs to retrieve an object**, without waiting for responses — e.g. in this example we pipeline 3 HTTP requests all together

Benefit: pipelining 3 requests → response time per file ≈ **1 RTT / 3**

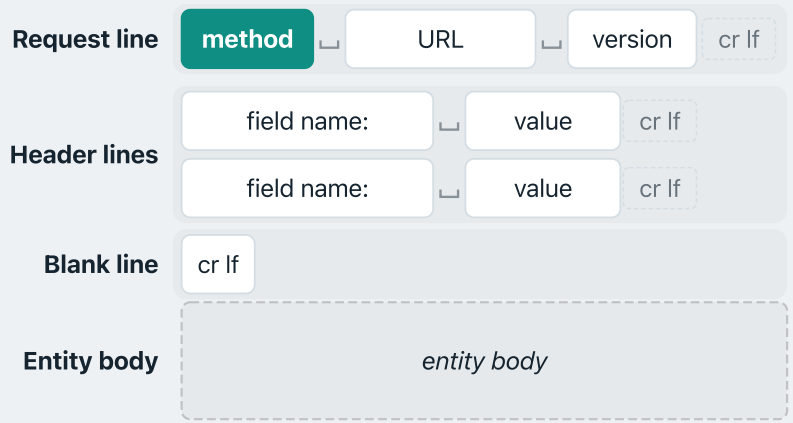


- Don't confuse this with TCP-level pipelining.
- **HTTP pipelining** operates at the **application layer**: the client sends several **HTTP requests** without waiting for each response.
- **TCP pipelining** operates at the **transport layer**: the sender keeps several **segments** in flight within its window. TCP cuts each HTTP message into segments, so these may be pieces of one HTTP request, or of several.

HTTP request message format

Method — what action to perform

Method	Action
GET	**Retrieve a resource**
HEAD	Retrieve headers only (no body)
POST	**Submit data to a resource**
PUT	Store / replace a resource
DELETE	**Remove a resource**

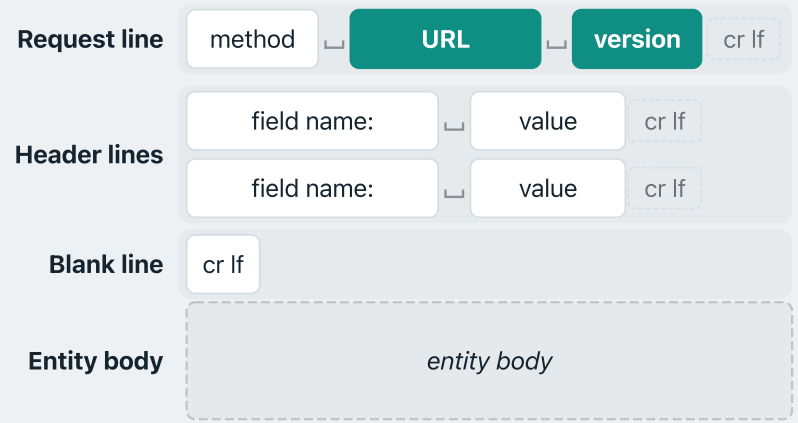


HTTP

HTTP request message format

URL: path to the requested resource — e.g. `/someDept/pic.gif`

HTTP Version: identifies the protocol in use — `HTTP/1.0` or `HTTP/1.1`



HTTP request message format

Header lines

- Zero or more `field-name: value` pairs, one per line
- Common examples:

```
Host: www.example.com
```

Mandatory in HTTP/1.1 — identifies the target server

```
Content-Type: text/html; charset=UTF-8
```

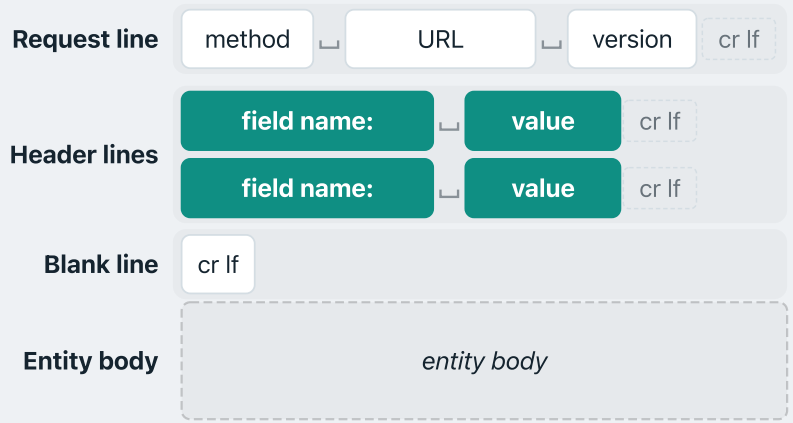
Format of the message body

```
Connection: close
```

Close the TCP connection after this response

```
Accept-Language: da, en-gb;q=0.8,
en;q=0.7
```

Preferred languages, weighted by q (default 1): Danish → British English → any English



HTTP request: an example

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language: fr

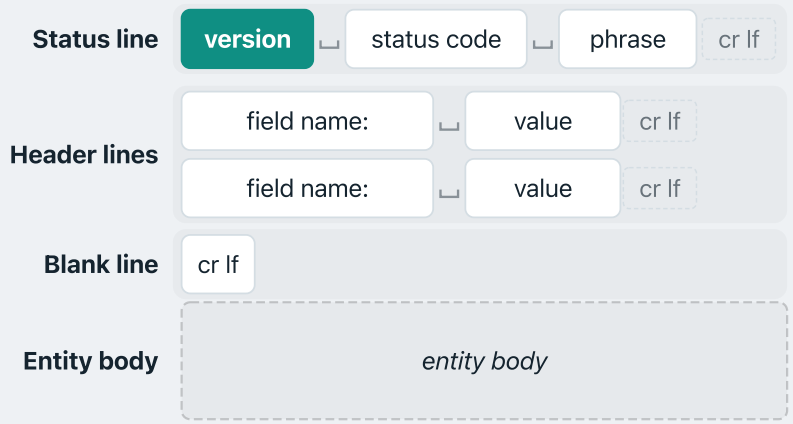
(empty entity body)
```

- **Method**: GET
- **URL**: /somedir/page.html
- **HTTP version**: HTTP/1.1
- **Header field**: Host: www.someschool.edu
- **Header field**: User-agent: Mozilla/4.0
- **Header field**: Connection: close
- **Header field**: Accept-language: fr
- **Body**: (empty)

HTTP response message format

Version

The HTTP version the server is using — e.g. `HTTP/1.1`, just as in the request line.

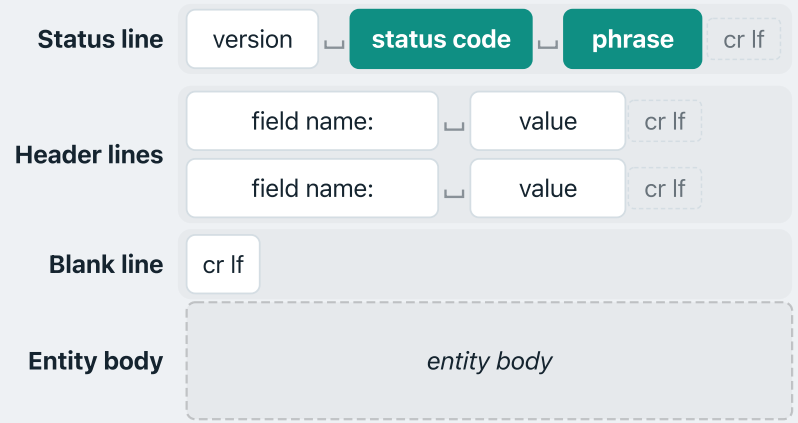


HTTP

HTTP response message format

Status code & phrase

- **Status code**: 3-digit number indicating the result of the request
- **Phrase**: human-readable text accompanying the code — e.g. `OK`, `Not Found`

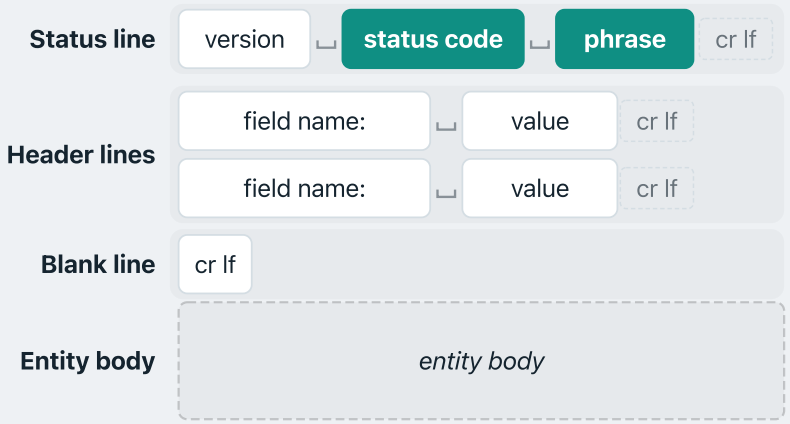


HTTP

HTTP response message format

Common examples

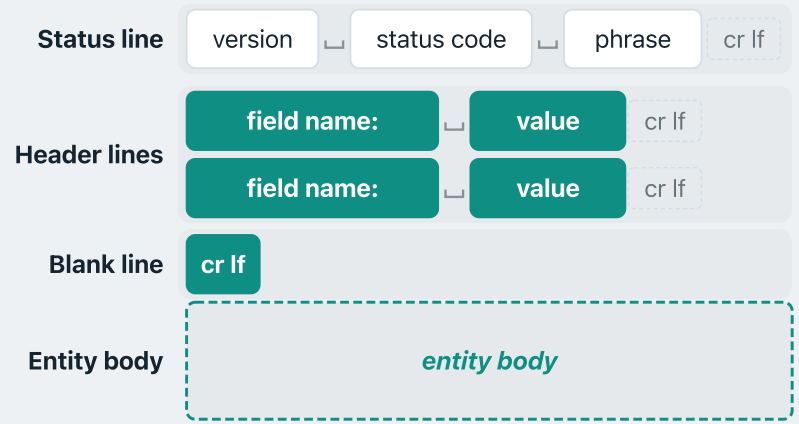
Code	Phrase
200	OK
301	Moved Permanently
400	Bad Request
404	Not Found
505	HTTP Version Not Supported



HTTP response message format

Header lines, blank line, entity body

Same structure as in the request: `field-name: value` header lines, a blank line (a lone CRLF), then the body — here, the **requested object** itself.



HTTP response: an example

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 06 Aug 1998 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 1998 …
Content-Length: 6821
Content-Type: text/html
```

```
<!DOCTYPE html>
<html><head><title>Example</title></head>
<body><h1>Hello!</h1>
<script>console.log("hi");</script>
</body></html>
```

- **Version**: HTTP/1.1
- **Status code**: 200 — success
- **Phrase**: OK
- **Header**: `Connection: close` — close after this response
- **Header**: `Date` — when the response was sent
- **Header**: `Server` — the server software
- **Header**: `Last-Modified` — when the object last changed
- **Header**: `Content-Length` — body size in bytes
- **Header**: `Content-Type` — the body is HTML
- **Body**: the requested HTML file (with embedded JS)

HTTP

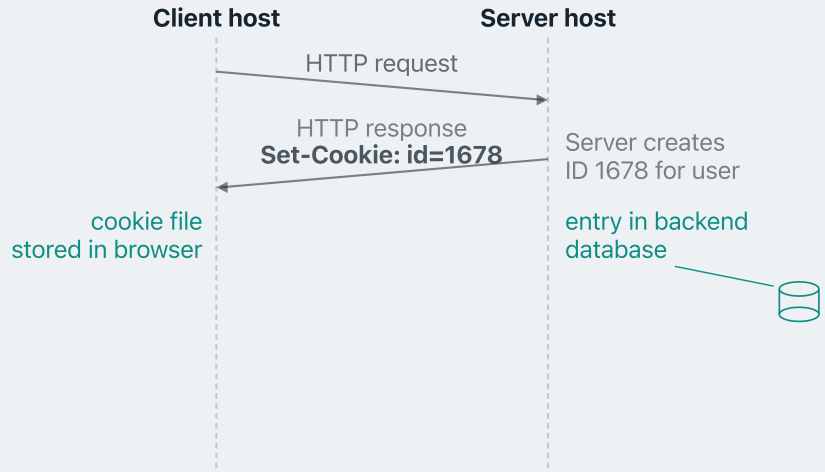
Cookies

> HTTP is **stateless** — every request is independent; the server retains no memory of past requests.
>
> So the server does not remember your earlier conversations with it. Yet many applications *need* that memory: when you refresh an online shop's page, the items in your cart should not just disappear. **How does the server know what is in your cart?**

The answer: **cookies** — here's how it works.

1. A `Set-Cookie` header line in the **HTTP response** — the server sends the client a unique ID on first contact
2. A `Cookie` header line in subsequent **HTTP requests** — the client echoes the ID back on every future request
3. A **cookie file** stored on the client, managed by the browser — persists across sessions
4. A **back-end database** on the server, keyed by the cookie ID — stores per-user state

Cookies: an example [RFC 6265]

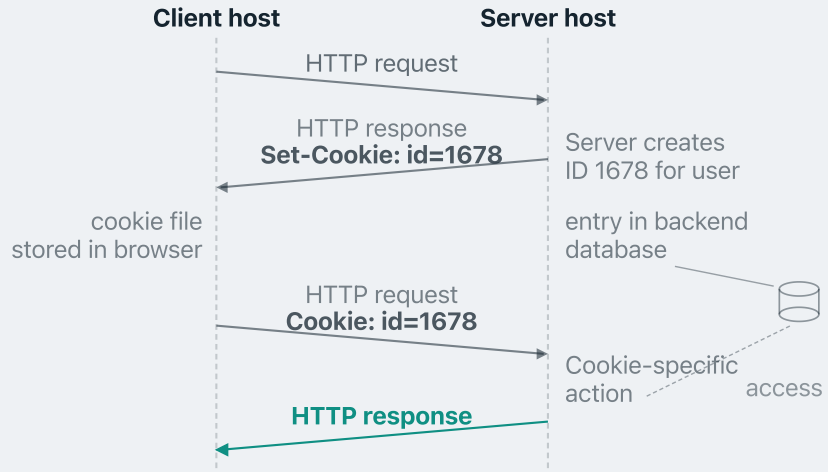


The client sends an HTTP request to the host

Set-Cookie: id=1678 — Server assigns a unique ID and sends it to the client

The cookie ID is stored as an entry in the back-end database, and in the client's browser

Cookies: an example [RFC 6265]



Cookie: id=1678 — Client attaches the ID on every future request to the same server

Server uses the ID to look up the user's state in its database

Server can then generate a **tailored response** — e.g. a personalised page, a pre-filled shopping cart — for the client's request

HTTP

One problem with the Web

Picture a university: thousands of users fetch the same popular pages and images, and every request travels across the Internet to the **origin server** that hosts the object.

Every one of those requests lands on the same origin server, through the same congested links — enough of them can **overwhelm** it.

How can we avoid this?

Web caching

A **web cache** (also called a **proxy server**) is a network entity that satisfies HTTP requests on behalf of an origin server — storing copies of recently requested objects locally.



Acts as a server

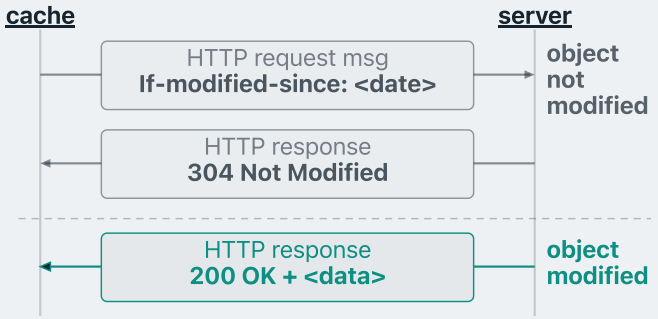
Receives HTTP requests from clients and, if it holds a copy of the object, returns it in the response — just like an origin server would

Acts as a client

When the object is not cached, it issues its own HTTP request to the origin server, stores the copy, then answers the client

Requests answered by the cache get a **quicker response**, and caching **reduces how often** requests have to go to the origin server.

Conditional GET



A **conditional GET** is an ordinary GET with an `If-modified-since` header line. A proxy uses it to check whether its cached copy is still up to date, e.g.:

```
GET /index.html HTTP/1.1
Host: www.example.com
If-modified-since: Mon, 10 Aug 2026 09:00:00 GMT
```

If the object is **unchanged**, server replies:

```
304 Not Modified
```

Object **changed**: server replies `200 OK` with the updated object in the body

MODULE 6

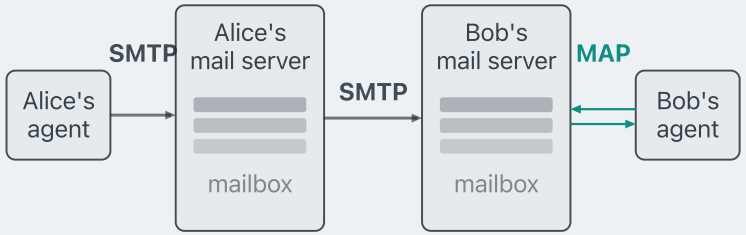
Email

Electronic Mail

1 · Network applications | 2 · The Web and HTTP | **3 · Email** | 4 · DNS

EMAIL

An overview of the email system



User agent (UA) — the app used to read, compose and send email (e.g. Outlook, Gmail, Apple Mail)

Mail server — a dedicated host that handles email on behalf of its users. It stores a **mailbox** per recipient for incoming mails and a **send queue** of outgoing mails

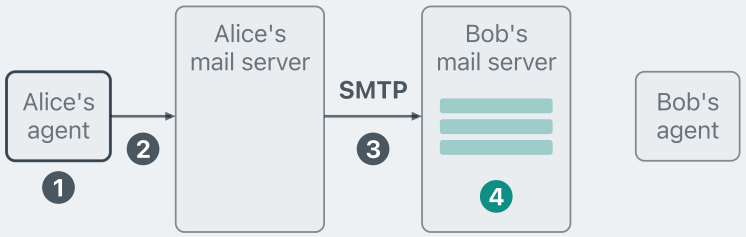
SMTP — Simple Mail Transfer Protocol; the sender's UA uses it to send a message to its own mail server

The sender's mail server then uses **SMTP** to transfer the message to the recipient's mail server across the internet

Mail access protocol (MAP) — used by UAs to retrieve messages from their mail server

SMTP in action

SMTP has a **client side** and a **server side**, and always runs over **TCP**.



① Alice uses her UA to **compose** the message

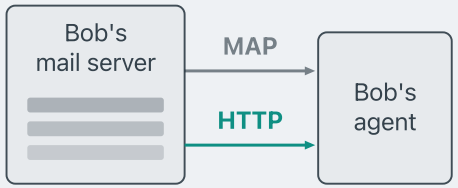
② Alice's UA (**SMTP client**) **sends** the message to her mail server (**SMTP server**)

③ The **SMTP client** on Alice's server opens a TCP connection to Bob's mail server and **transfers** Alice's message over it

④ The **SMTP server** on Bob's server receives the message and places it in Bob's **mailbox**

EMAIL

Retrieving email from the server



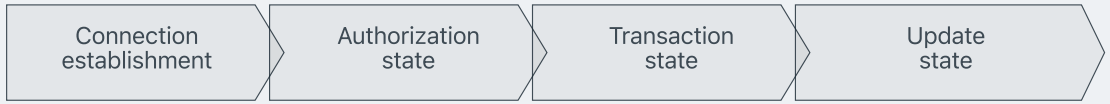
The recipient's UA retrieves email from its mail server either with a **mail access protocol (MAP)**, e.g.

- **POP3** — Post Office Protocol version 3 [RFC 1939]
- **IMAP** — Internet Message Access Protocol [RFC 9051]

or through **web-based email**, where the UA is a web browser that uses **HTTP** to reach the mail server

POP3

How POP3 works



Connection establishment	Authorization state	Transaction state	Update state
UA establishes a **TCP connection** to the mail server on **port 110**	UA sends **username** and **password** (cleartext) to authenticate	UA **retrieves messages**; can mark for deletion, remove marks, obtain statistics	Server **deletes** marked messages and **closes** the connection

> Messages wait on the server until the UA downloads them

Notable limitations:

- Keeps no state on the server (e.g. read or unread), so what you do on one device is not seen on the others
- Does not support server-side folder management — users cannot organise emails into folders on the server

IMAP

Internet Message Access Protocol

IMAP addresses POP3's limitations:

> IMAP **keeps per-message state on the server**, e.g. read or unread, flagged, so every device sees the same status

> It supports **folders on the server**: create folders and move messages into them, visible from any device

WEB-BASED EMAIL

Accessing email via a browser

Web-based email is accessed through a web browser

The mailbox stays on the server; reading an email is like browsing the Web — the browser fetches each message as a web page

Which application-layer protocol does the browser use to talk to the mail server?

HTTP, the same protocol behind every other web page

Web-based email services

Examples: **Gmail**, **Outlook** (web)

EMAIL

Comparing POP3, IMAP and web-based email

	POP3	IMAP	Web-based email
Accessed with	A mail app	A mail app	A web browser, over **HTTP**
Messages	Downloaded to the device; deleted from the server, or optionally kept	Kept on the server	Kept on the server
Folders	None on the server; a single mailbox	Created and kept on the server	Created and kept on the server
Multiple devices	Not synced: read status and folders exist only on each device	Read status and folders stay in sync across devices	Any device with a browser sees the same mailbox

MESSAGE FORMAT

Structure of an email message

An email message, as carried by **SMTP**, consists of **header fields**, followed (optionally) by a **body** [RFC 5322]

Header fields describe the message, e.g. `From:` and `Subject:`

The **body** carries the message content itself

SMTP requires the whole message to be **ASCII text**, which encodes English letters, digits and punctuation

Problem: ASCII alone cannot carry images, attachments, or non-ASCII characters (e.g. Chinese, Arabic)

Header fields

```
Date: Mon, 10 Aug 2026 09:00 GMT
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
Cc: charlie@example.com
```

Body

```
Hello Bob, here is the picture
you asked for. Enjoy!
```

MIME

Multipurpose Internet Mail Extensions

MIME extends the message format with **additional header fields**

These fields declare the content's **type** and how it is **encoded**

so the receiver knows how to **decode the body**

e.g. **base64** encoding converts binary data into printable ASCII text, which can be used for image attachments

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg
---------------------------------
(base64 encoded data…
…base64 encoded data)
```

→



decoded image

MODULE 6

DNS

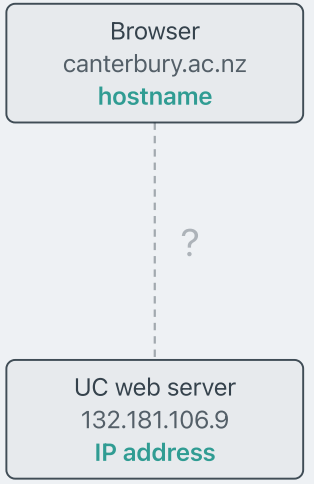
Domain Name System

1 · Network applications | 2 · The Web and HTTP | 3 · Email | **4 · DNS**

DNS

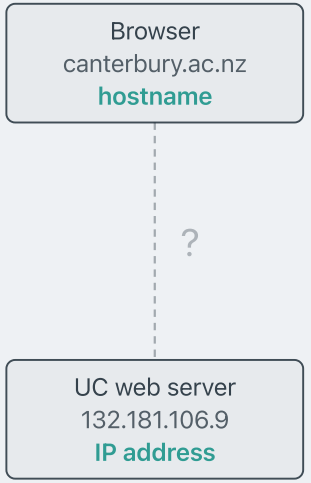
Hostnames and IP addresses

- As we have seen, network routing entirely relies on **IP addresses** — numerical identifiers, e.g. `132.181.106.9`
- However, when you browse the web, you typically type a **human-friendly hostname** such as `canterbury.ac.nz`
- How does the browser know how to translate between a hostname and an IP address?



DNS

Domain Name System — overview



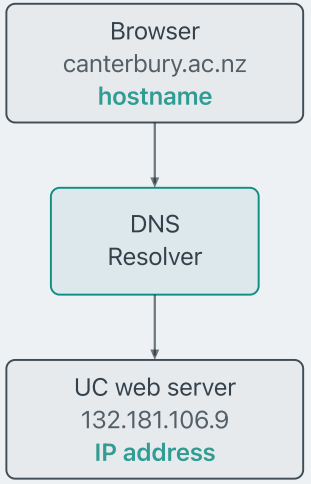
Domain Name System — overview

DNS is that translation mechanism — it maps human-readable **hostnames** to numerical **IP addresses**

DNS stores this mapping in a **distributed database**, spread across a hierarchy of DNS servers [RFC 1034, 1035]

DNS is an **application-layer protocol** running mainly over **UDP on port 53** (TCP port 53 for large responses and zone transfers)

Why UDP? A query and its reply each fit in one small datagram, so there is no connection setup delay, providing a quick response.



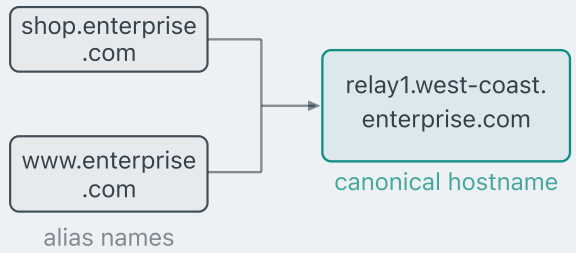
hostname → IP address

DNS SERVICES

Additional services provided by DNS

Host aliasing — a host may have a cumbersome **canonical hostname**, e.g. `relay1.west-coast.enterprise.com`, which is hard to remember.

With host aliasing, it can also be reached via simpler **alias names**, e.g. `www.enterprise.com` and `shop.enterprise.com`; DNS maps each alias to the canonical hostname.



DNS SERVICES

Additional services provided by DNS

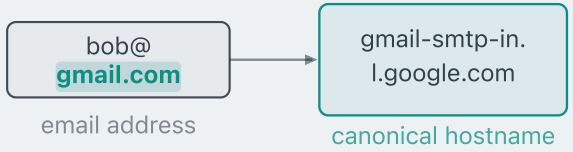
Host aliasing — a host may have a cumbersome **canonical hostname**, e.g. `relay1.west-coast.enterprise.com`, which is hard to remember.

With host aliasing, it can also be reached via simpler **alias names**, e.g. `www.enterprise.com` and `shop.enterprise.com`; DNS maps each alias to the canonical hostname.

Mail server aliasing — the same idea for email.

e.g. the mail server that handles Gmail has the cumbersome canonical hostname `gmail-smtp-in.l.google.com`.

Yet Bob's email address can simply be `bob@gmail.com`: DNS uses the part after the @, `gmail.com`, to find that mail server.



Additional services by DNS: load distribution

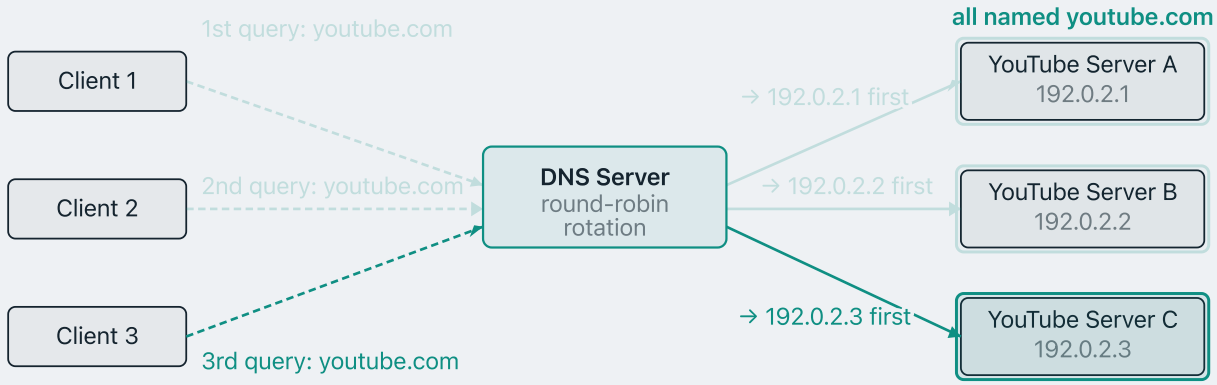
Many popular web services — e.g. YouTube, Google, Netflix — are hit by **millions of users** simultaneously. A **single server** cannot handle that volume.

The service is therefore distributed across **many server instances**, each with a **distinct IP address**, all sharing the **same hostname** (e.g. `youtube.com`).

How are users assigned to these servers?

DNS load balancing — DNS stores all the IPs and **rotates their order** on each response, so consecutive clients get a different IP listed first.

e.g. three clients query `youtube.com` in turn:



DNS

DNS is distributed, not centralised

Recall that DNS stores the hostname-to-IP mapping in a **distributed database**, spread across many DNS servers.

Why not a centralised design — a single server hosting the whole database, so that every DNS query goes to it?

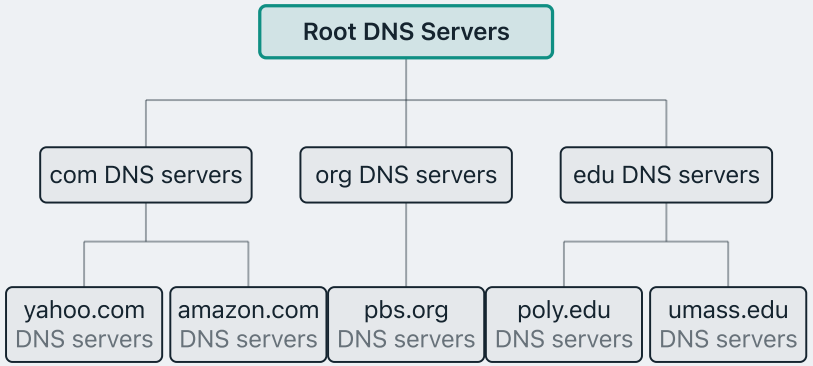
Single point of failure
If the one server crashes, DNS lookup fails for the entire Internet

Traffic volume
Billions of DNS queries per day — no single machine can handle that load

Distant centralised database
All queries must travel to one location — users far away face high latency on every lookup

Maintenance
Billions of hostname records change constantly — impossible to keep one database current

A hierarchical distributed database

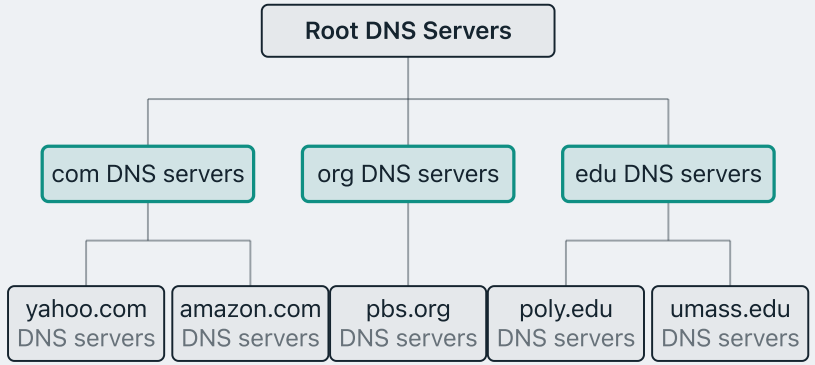


Root DNS Servers

- The top of the hierarchy: looking up a hostname from scratch **starts here**
- There are 13 root server identities (a–m.root-servers.net)
- Each is replicated: its IP address is shared by many machines worldwide, and a query goes to the nearest one

DNS

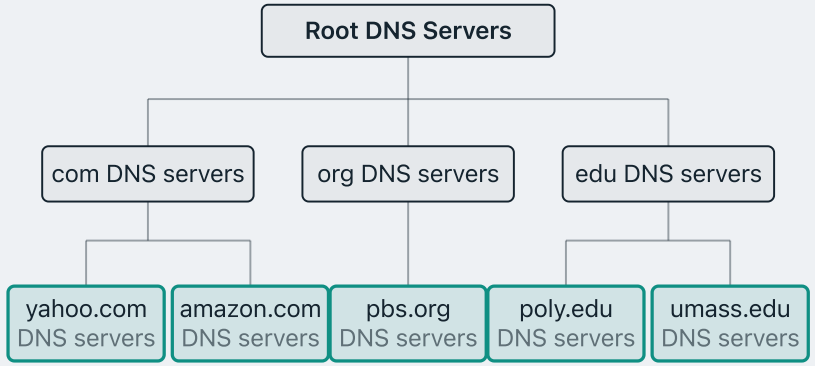
A hierarchical distributed database



Top-Level Domain (TLD) DNS Servers

- Handle generic domains: **.com**, **.org**, **.edu**
- ...and country-code domains: **.nz**, **.uk**
- Each is run by a dedicated registry, e.g. **Verisign** (.com), **InternetNZ** (.nz)

A hierarchical distributed database



Authoritative DNS Servers

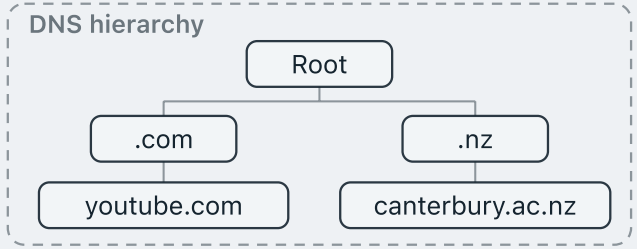
- Each organisation's own DNS server
- It stores the organisation's own records — the original, **definitive** copy that every answer for its hostnames comes from
- e.g. Canterbury's authoritative server holds the IP for `www.canterbury.ac.nz`

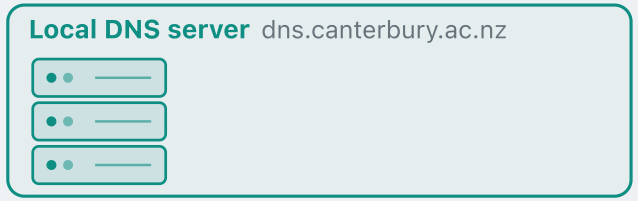
DNS ARCHITECTURE

Local DNS server and DNS caching

Every ISP or institution operates a **local DNS server** (also called a *default name server*).

It is **not** part of the root / TLD / authoritative hierarchy — it sits **separately** from that.

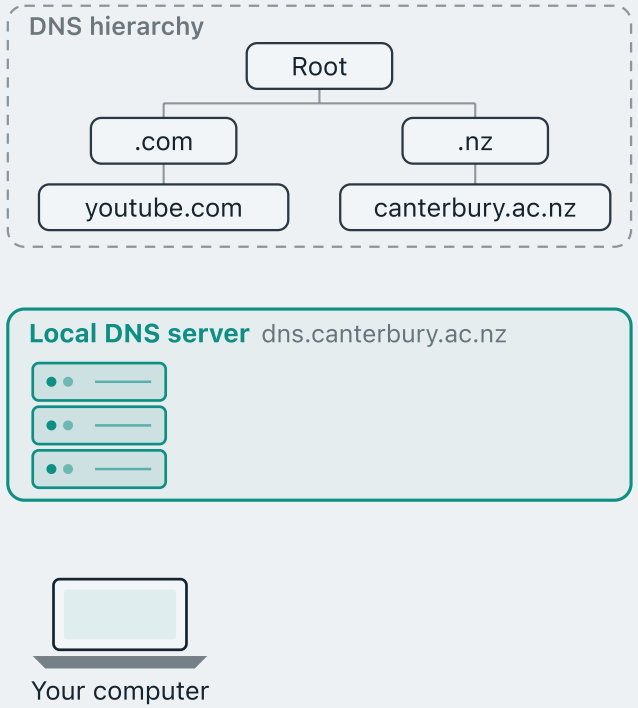




DNS ARCHITECTURE

Local DNS server and DNS caching

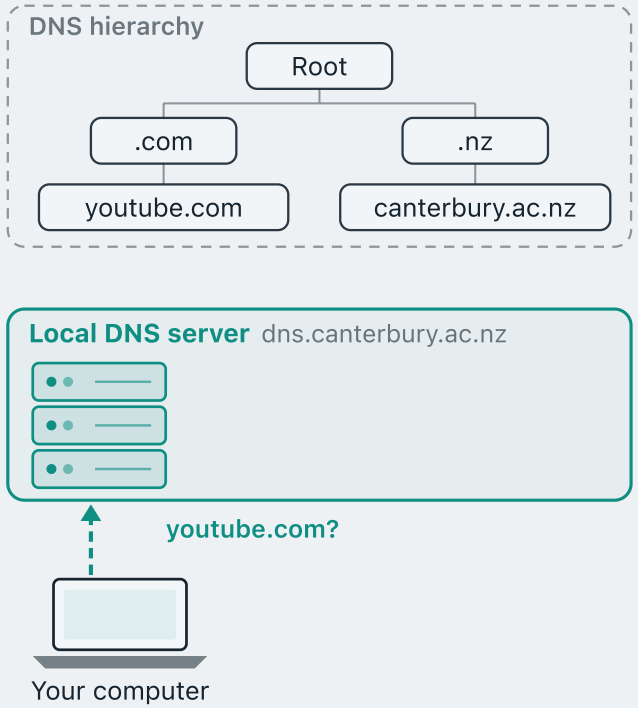
When you join the network, your device is automatically configured to use one — e.g. the University of Canterbury runs its own local DNS server for all campus users.



DNS ARCHITECTURE

Local DNS server and DNS caching

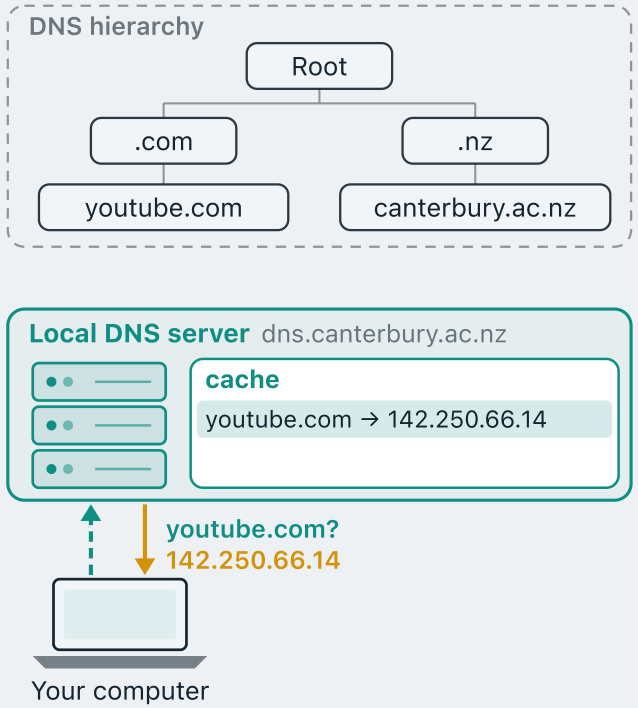
When your machine makes a DNS lookup, the query goes **first to the local DNS server**.



DNS ARCHITECTURE

Local DNS server and DNS caching

If the answer is cached: the local DNS server replies **immediately** — no hierarchy lookup needed.



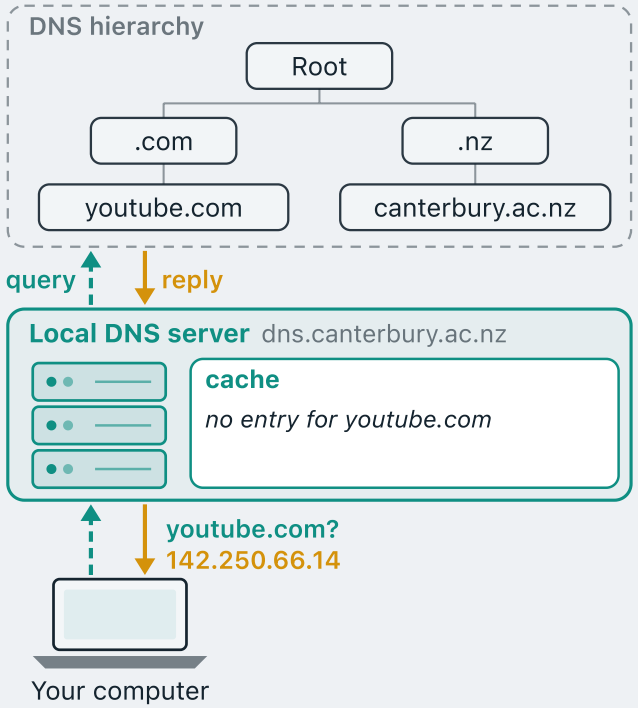
DNS ARCHITECTURE

Local DNS server and DNS caching

If not cached: the local server **forwards the query into the hierarchy**.

The hierarchy replies with the IP address.

The local server **relays the reply** back to you,



DNS ARCHITECTURE

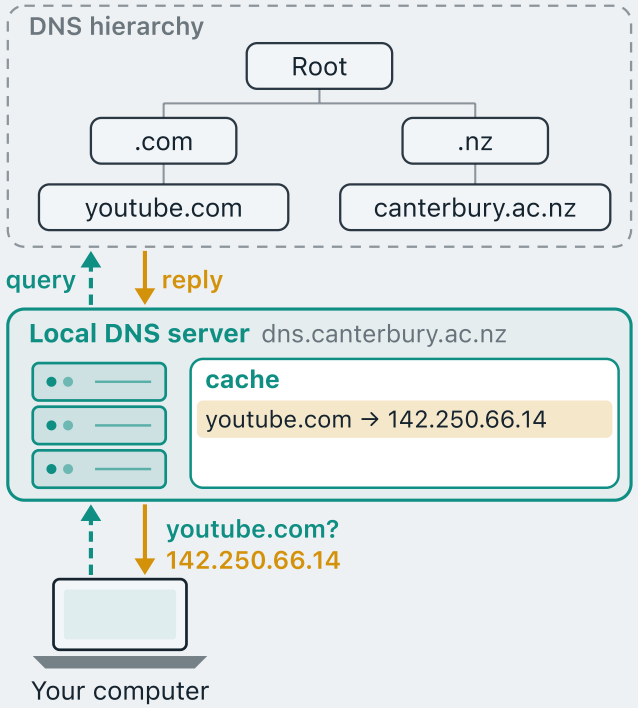
Local DNS server and DNS caching

If not cached: the local server **forwards the query into the hierarchy**.

The hierarchy replies with the IP address.

The local server **relays the reply** back to you,

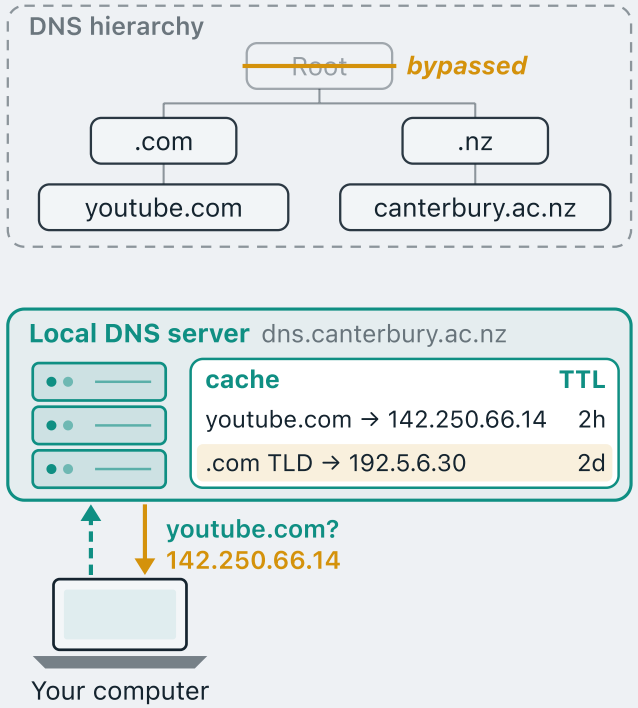
and **caches the hostname-to-IP mapping** for future queries.



DNS ARCHITECTURE

Local DNS server and DNS caching

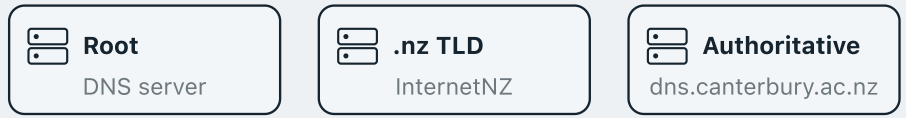
Cache entries have a **TTL (time-to-live)** and expire after some time (typically hours). TLD server addresses are routinely cached in local DNS servers — so **root servers are bypassed for most everyday queries**.

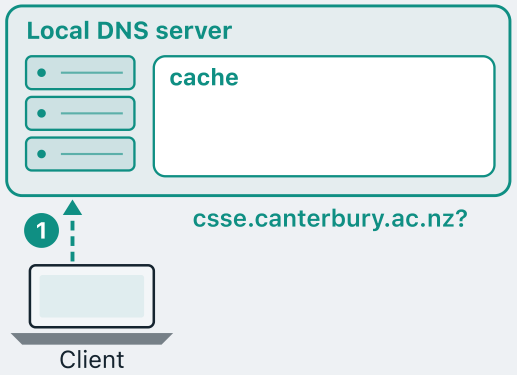


DNS ARCHITECTURE

Resolving a hostname: the full journey

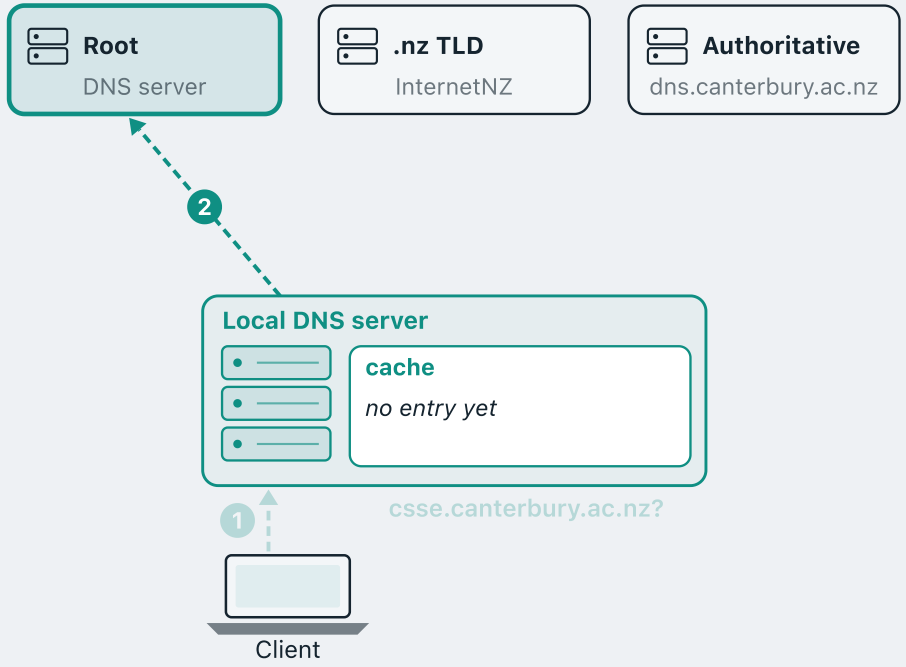
Step 1. A client wants to resolve **csse.canterbury.ac.nz**. It sends a DNS query to its **local DNS server**.





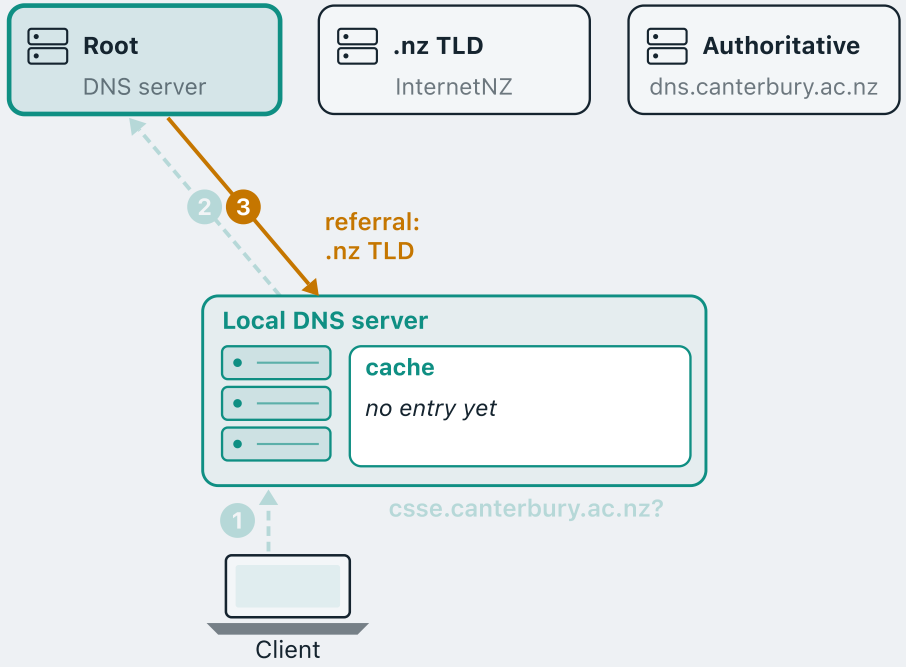
Resolving a hostname: the full journey

Step 2. The local DNS doesn't have the answer cached. It contacts a **root DNS server**.



Resolving a hostname: the full journey

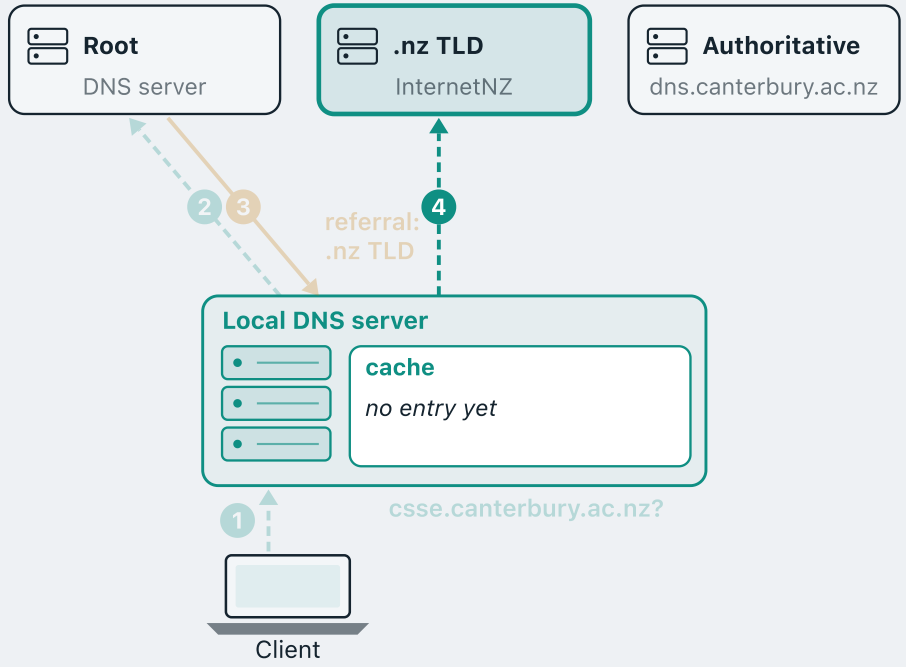
Step 3. The root server returns a **referral**: "I don't know — try a **.nz TLD** server" and gives its address.



DNS ARCHITECTURE

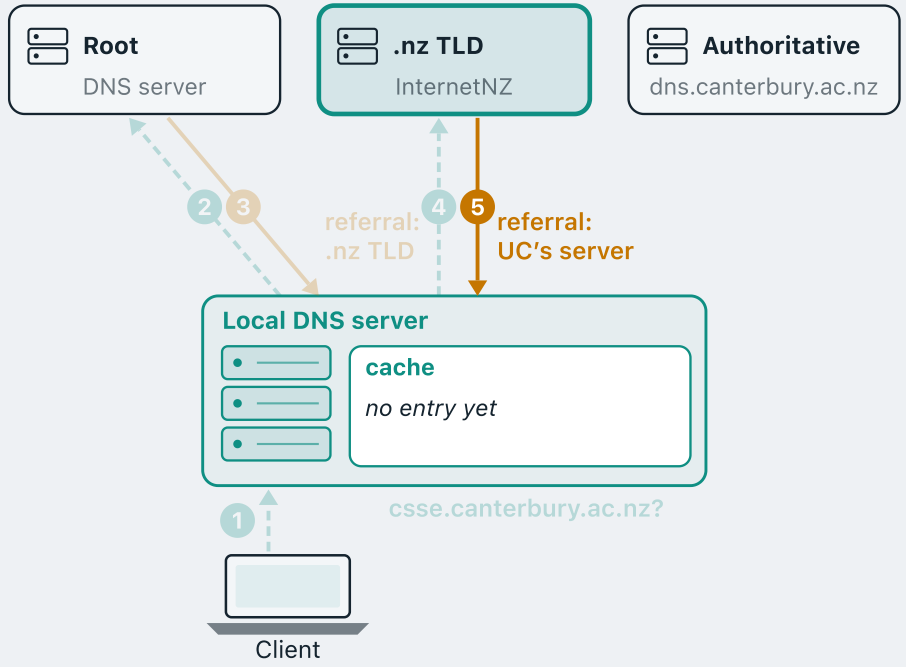
Resolving a hostname: the full journey

Step 4. The local DNS follows the referral and contacts the **.nz TLD server**.



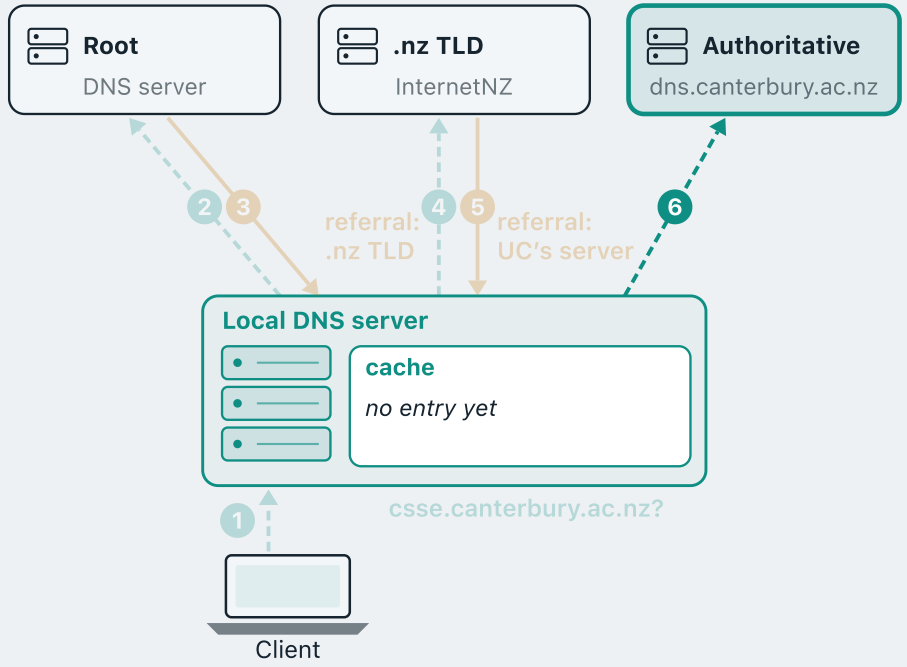
Resolving a hostname: the full journey

Step 5. The .nz TLD returns another **referral**: "Try **dns.canterbury.ac.nz** — the authoritative server for canterbury.ac.nz."



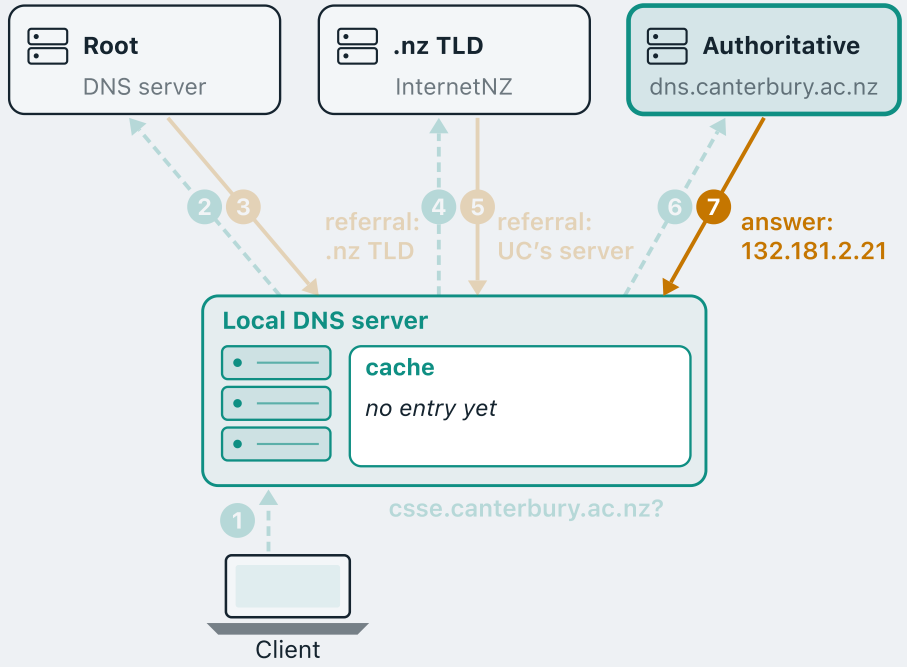
Resolving a hostname: the full journey

Step 6. The local DNS contacts **dns.canterbury.ac.nz** — the authoritative name server for the canterbury.ac.nz zone.



Resolving a hostname: the full journey

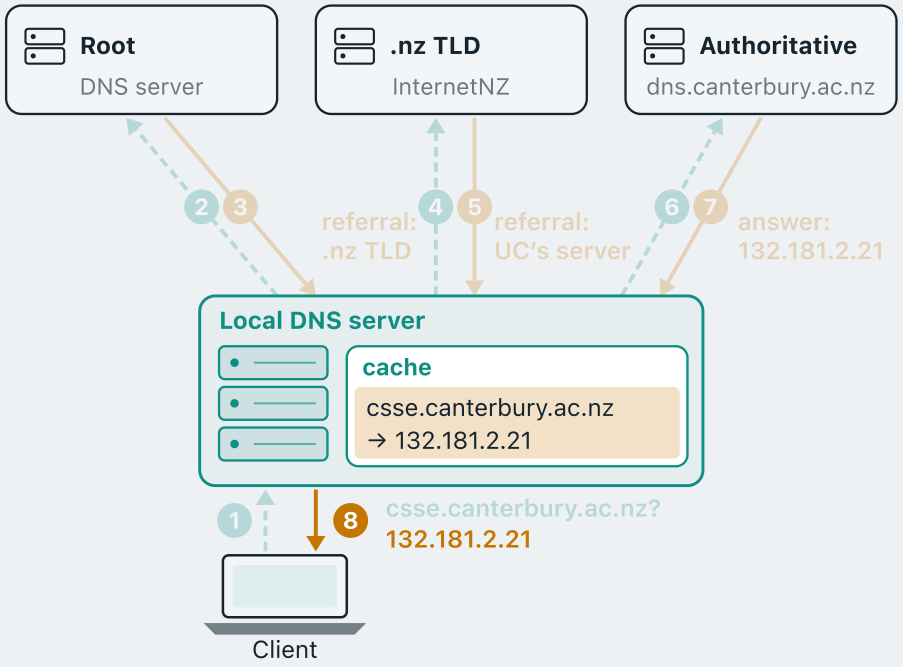
Step 7. The authoritative server has the answer! It replies: "csse.canterbury.ac.nz = **132.181.2.21**"



DNS ARCHITECTURE

Resolving a hostname: the full journey

Step 8. The local DNS returns **132.181.2.21** to the client — and **caches** the mapping for future queries.

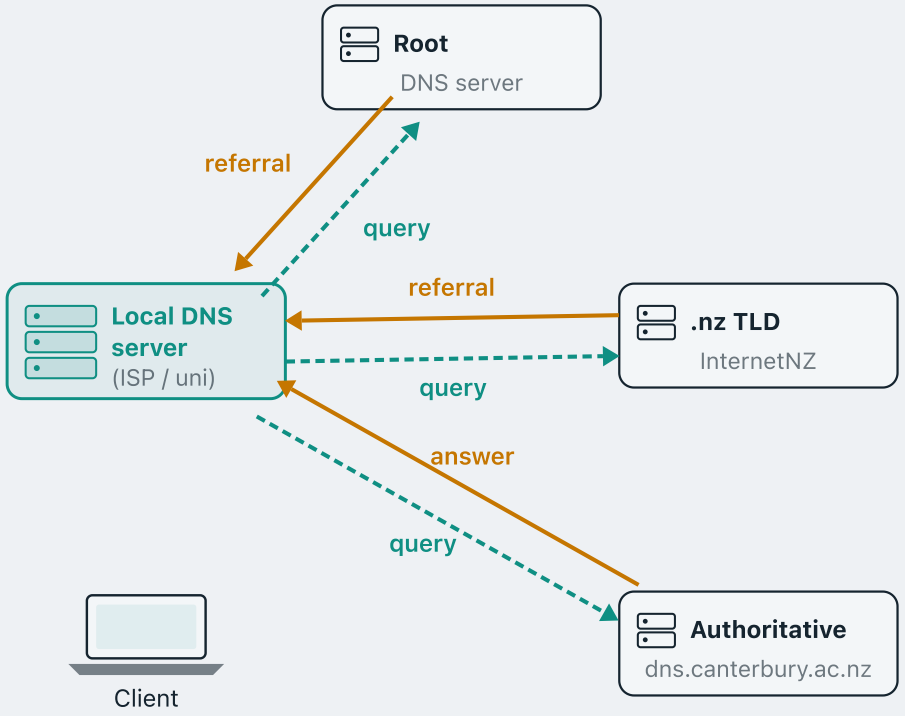


DNS ARCHITECTURE

Recursive vs iterative queries

Iterative query

- The contacted server does **not** query other servers for you: it replies only with what it knows itself.
- If it doesn't know the answer, it returns a **referral**: "I don't know — try that server."
- The authoritative server *does* know, so its reply to the same kind of query is the **answer**.

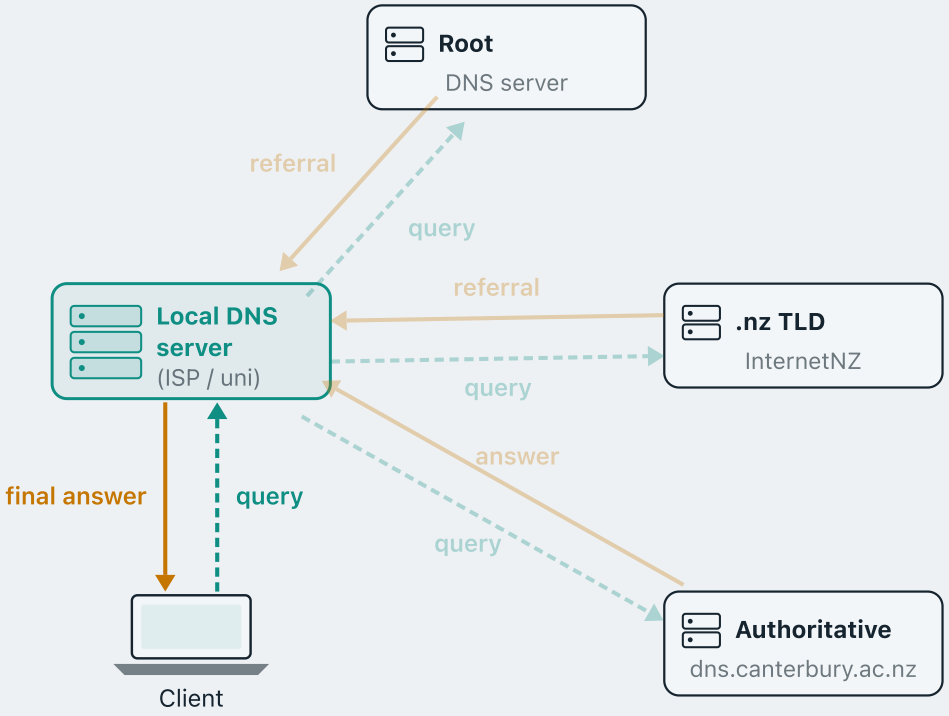


DNS ARCHITECTURE

Recursive vs iterative queries

Recursive query

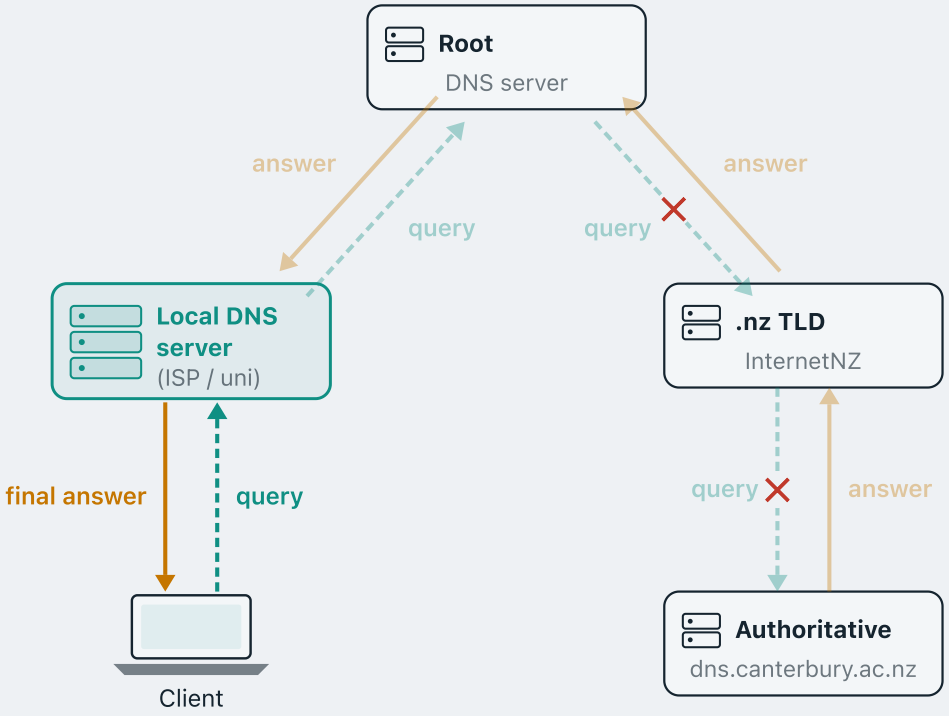
- The contacted server takes **full responsibility**:
- it resolves the name itself, querying the next servers on your behalf,
- and returns the **final answer**. e.g. client → local DNS server.



Recursive vs iterative queries

Can we go fully recursive?

- In principle, yes: root asks the TLD server, which asks the authoritative server,
- and the answer flows back along the chain.
- In practice, root and TLD servers **refuse recursion**: too much load.



DNS ARCHITECTURE

DNS Resource Records

DNS's distributed database stores its mappings as **Resource Records (RRs)**: each DNS server keeps a table of records, all in the same four-field format.

name	**value**	**type**	**TTL**
e.g. a hostname	e.g. an IP address	A, NS, CNAME, MX	time-to-live (s)

Type A

name = hostname · **value** = IPv4 address

NAME	VALUE	TYPE	TTL (s)
web.canterbury.ac.nz	132.181.106.9	A	3600
itdns.canterbury.ac.nz	132.181.2.225	A	3600
mail.canterbury.ac.nz	132.181.3.225	A	3600
canterbury.ac.nz	itdns.canterbury.ac.nz	NS	86400
www.canterbury.ac.nz	web.canterbury.ac.nz	CNAME	3600
canterbury.ac.nz	mail.canterbury.ac.nz	MX	3600

DNS ARCHITECTURE

DNS Resource Records

DNS's distributed database stores its mappings as **Resource Records (RRs)**: each DNS server keeps a table of records, all in the same four-field format.

name	**value**	**type**	**TTL**
e.g. a hostname	e.g. an IP address	A, NS, CNAME, MX	time-to-live (s)

Type NS

name = domain · **value** = hostname of the authoritative DNS server for that domain

"I don't know — but here's the name of the server that does."

NAME	VALUE	TYPE	TTL (s)
web.canterbury.ac.nz	132.181.106.9	A	3600
itdns.canterbury.ac.nz	132.181.2.225	A	3600
mail.canterbury.ac.nz	132.181.3.225	A	3600
canterbury.ac.nz	itdns.canterbury.ac.nz	NS	86400
www.canterbury.ac.nz	web.canterbury.ac.nz	CNAME	3600
canterbury.ac.nz	mail.canterbury.ac.nz	MX	3600

DNS ARCHITECTURE

DNS Resource Records

DNS's distributed database stores its mappings as **Resource Records (RRs)**: each DNS server keeps a table of records, all in the same four-field format.

- **name** — e.g. a hostname
- **value** — e.g. an IP address
- **type** — A, NS, CNAME, MX
- **TTL** — time-to-live (s)

Type CNAME

name = alias hostname · **value** = canonical (real) hostname it maps to

NAME	VALUE	TYPE	TTL (s)
web.canterbury.ac.nz	132.181.106.9	A	3600
itdns.canterbury.ac.nz	132.181.2.225	A	3600
mail.canterbury.ac.nz	132.181.3.225	A	3600
canterbury.ac.nz	itdns.canterbury.ac.nz	NS	86400
www.canterbury.ac.nz	web.canterbury.ac.nz	CNAME	3600
canterbury.ac.nz	mail.canterbury.ac.nz	MX	3600

DNS ARCHITECTURE

DNS Resource Records

DNS's distributed database stores its mappings as **Resource Records (RRs)**: each DNS server keeps a table of records, all in the same four-field format.

name	**value**	**type**	**TTL**
e.g. a hostname	e.g. an IP address	A, NS, CNAME, MX	time-to-live (s)

Type MX

name = domain · **value** = hostname of the mail server for that domain

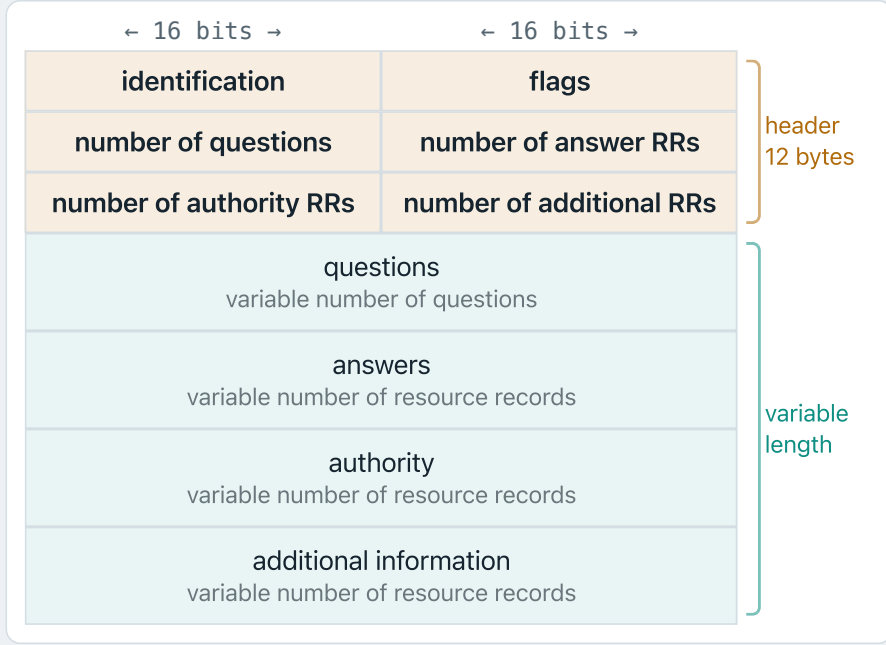
Unlike CNAME, it only redirects mail: a sending mail server (**SMTP**) looks it up to find where to deliver a domain's email.

NAME	VALUE	TYPE	TTL (s)
web.canterbury.ac.nz	132.181.106.9	A	3600
itdns.canterbury.ac.nz	132.181.2.225	A	3600
mail.canterbury.ac.nz	132.181.3.225	A	3600
canterbury.ac.nz	itdns.canterbury.ac.nz	NS	86400
www.canterbury.ac.nz	web.canterbury.ac.nz	CNAME	3600
canterbury.ac.nz	mail.canterbury.ac.nz	MX	3600

DNS ARCHITECTURE

DNS protocol message format

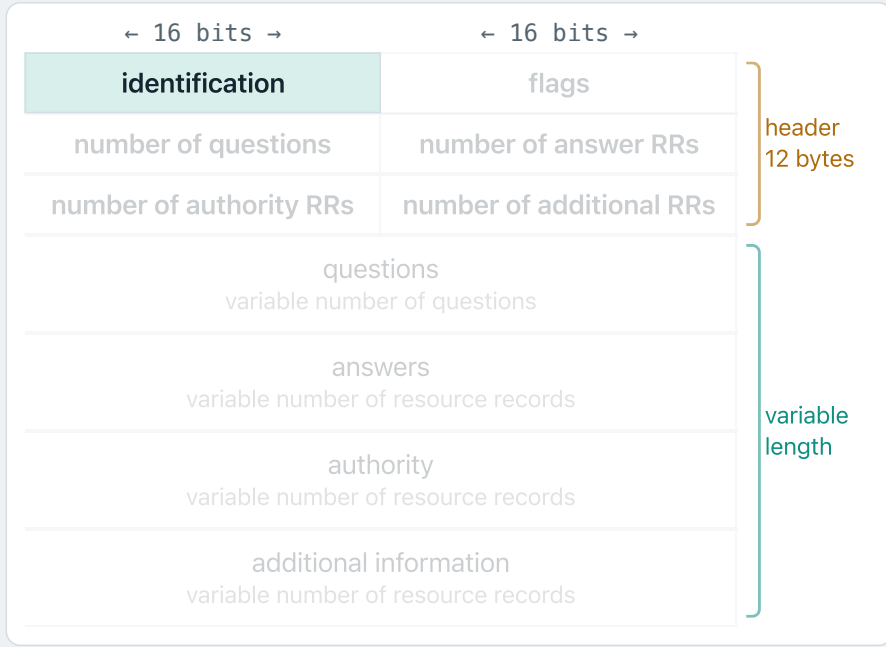
DNS uses the **same message format** for both **queries** and **replies** — a fixed 12-byte header followed by four variable-length sections.



DNS protocol message format

Identification

16-bit number assigned by the client. The reply copies this same value, so the client can match each reply to its original query.

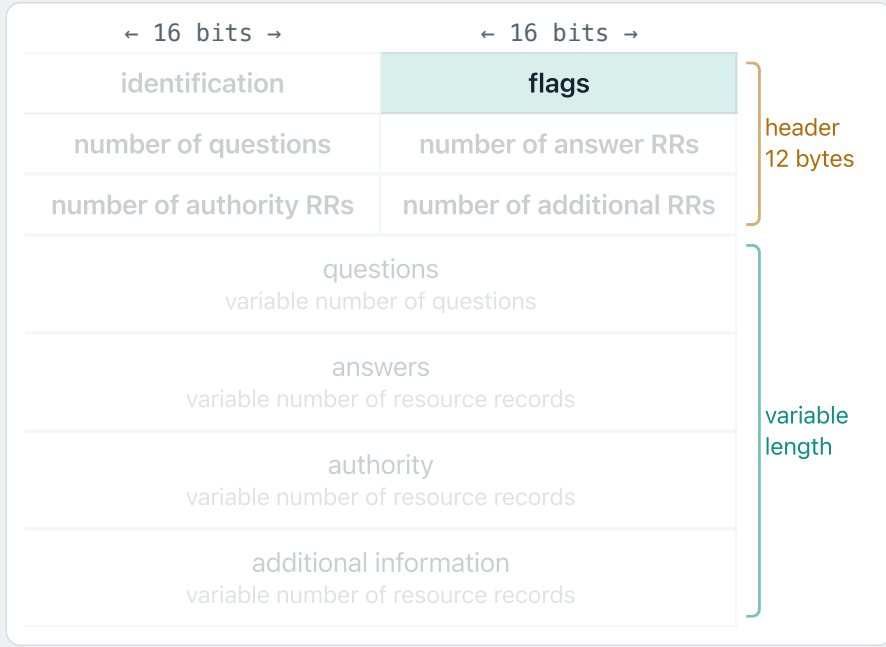


DNS ARCHITECTURE

DNS protocol message format

Flags

- Query (0) or reply (1)
- Recursion desired (RD) — client requests full recursive resolution
- Recursion available (RA) — server indicates it supports recursion

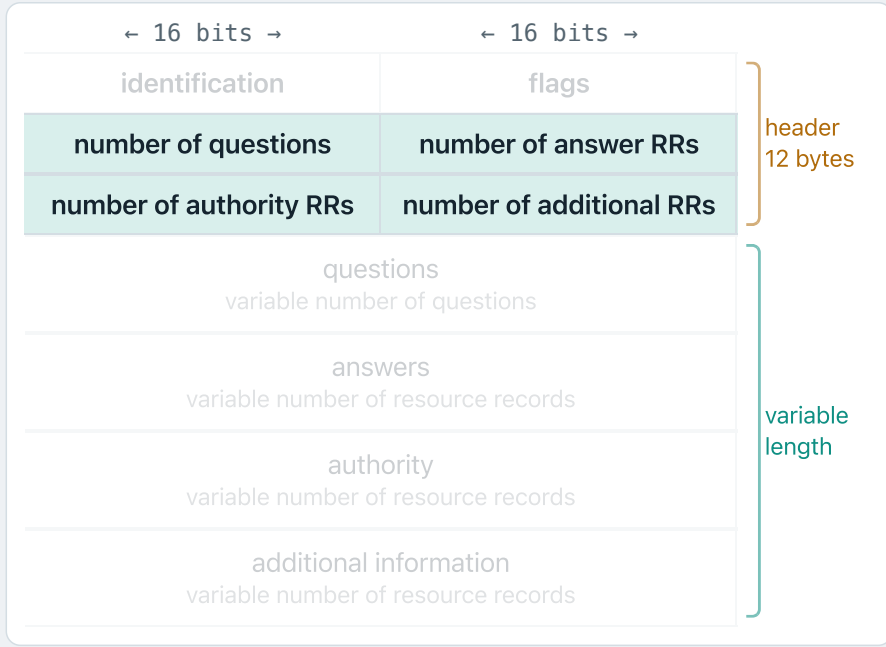


DNS ARCHITECTURE

DNS protocol message format

Count fields

Four 16-bit counters — one per section below — telling how many records appear in each variable-length section.

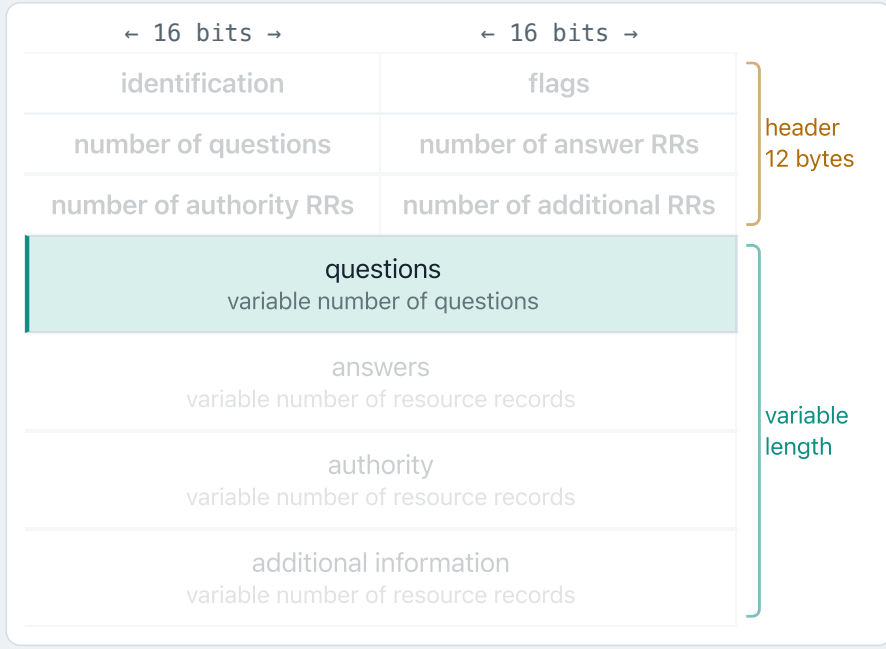


DNS protocol message format

Questions

The name and record type being queried. e.g. this Type A question asks for the IP address of the domain facebook.com.

```
(facebook.com, A)
```

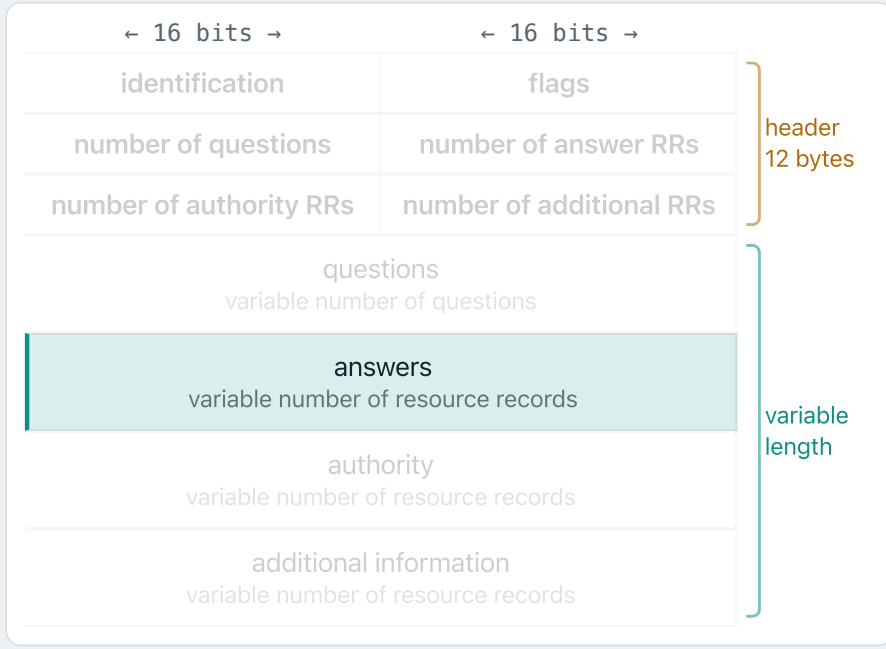


DNS protocol message format

Answers

Resource Records (RRs) that directly answer the query.

```
(facebook.com, 157.240.22.35, A, TTL=300)
```

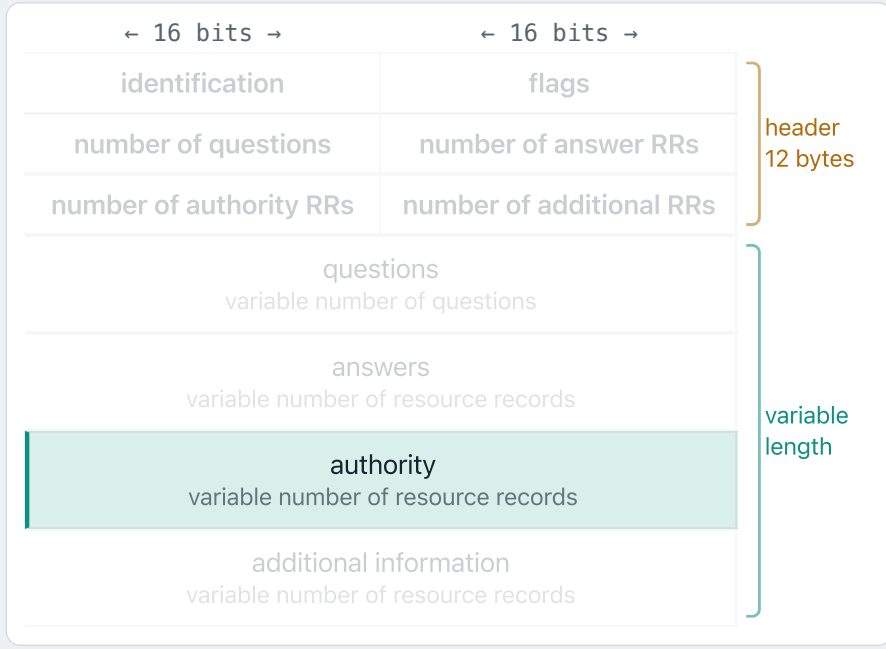


DNS protocol message format

Authority

NS records pointing to authoritative servers — returned in referral responses.

```
(facebook.com, a.ns.facebook.com, NS)
```

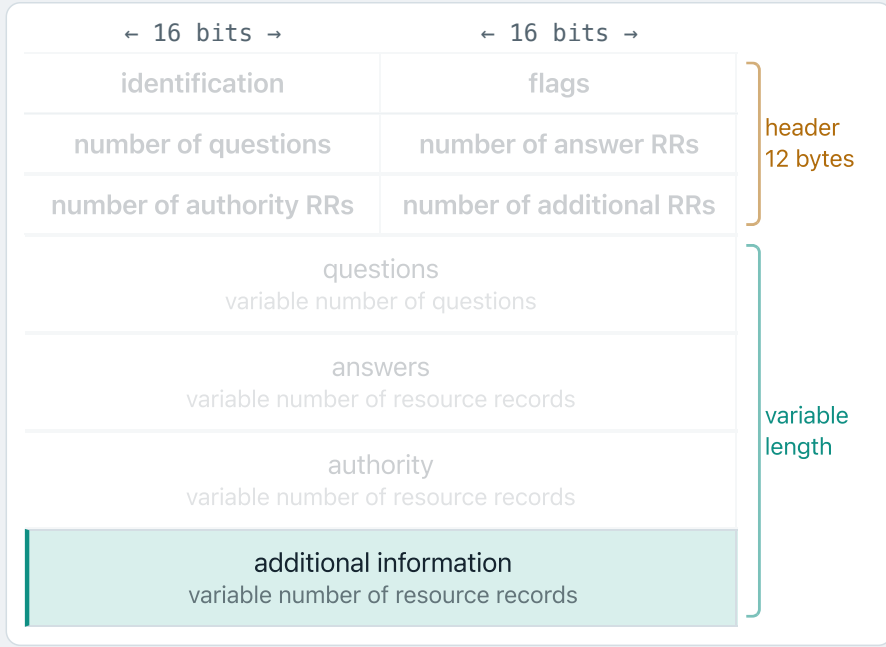


DNS protocol message format

Additional

'Glue' Resource Records (RRs) accompanying the authority section — so you can reach the name server without an extra lookup.

```
(a.ns.facebook.com, 129.134.30.12, A)
```



Query & response: an example

QUERY client → local DNS · UDP 53

```
Transaction ID  0x6492
Flags           0x0100 (query, RD=1)
Questions  1    Answer RRs  0
Authority RRs  0    Additional RRs  0
```

QUESTIONS

```
www.canterbury.ac.nz    A
```

The question

The query asks one question: what is the IP address (type **A**) of `www.canterbury.ac.nz`?

RESPONSE local DNS → client

```
Transaction ID  0x6492
Flags           0x8180 (reply, RA=1, no error)
Questions  1    Answer RRs  2
Authority RRs  1    Additional RRs  1
```

QUESTIONS

```
www.canterbury.ac.nz    A
```

ANSWERS

```
www.canterbury.ac.nz    CNAME
  web.canterbury.ac.nz
web.canterbury.ac.nz    A
  132.181.106.9
```

AUTHORITY

```
canterbury.ac.nz    NS
  itdns.canterbury.ac.nz
```

ADDITIONAL

```
itdns.canterbury.ac.nz    A
  132.181.2.225
```

Query & response: an example

QUERY client → local DNS · UDP 53

```
Transaction ID  0x6492
Flags           0x0100 (query, RD=1)
Questions  1    Answer RRs  0
Authority RRs  0    Additional RRs  0
```

QUESTIONS

```
www.canterbury.ac.nz    A
```

The answer

The response answers with **two records**.

RESPONSE local DNS → client

```
Transaction ID  0x6492
Flags           0x8180 (reply, RA=1, no error)
Questions  1    Answer RRs  2
Authority RRs  1    Additional RRs  1
```

QUESTIONS

```
www.canterbury.ac.nz    A
```

ANSWERS

```
www.canterbury.ac.nz    CNAME
  web.canterbury.ac.nz
web.canterbury.ac.nz    A
  132.181.106.9
```

AUTHORITY

```
canterbury.ac.nz    NS
  itdns.canterbury.ac.nz
```

ADDITIONAL

```
itdns.canterbury.ac.nz    A
  132.181.2.225
```

Query & response: an example

QUERY client → local DNS · UDP 53

```
Transaction ID  0x6492
Flags           0x0100 (query, RD=1)
Questions  1    Answer RRs  0
Authority RRs  0    Additional RRs  0
```

QUESTIONS

```
www.canterbury.ac.nz    A
```

First: a CNAME

`www.canterbury.ac.nz` is an **alias**; its canonical name is `web.canterbury.ac.nz`.

RESPONSE local DNS → client

```
Transaction ID  0x6492
Flags           0x8180 (reply, RA=1, no error)
Questions  1    Answer RRs  2
Authority RRs  1    Additional RRs  1
```

QUESTIONS

```
www.canterbury.ac.nz    A
```

ANSWERS

```
www.canterbury.ac.nz    CNAME
  web.canterbury.ac.nz
web.canterbury.ac.nz    A
  132.181.106.9
```

AUTHORITY

```
canterbury.ac.nz    NS
  itdns.canterbury.ac.nz
```

ADDITIONAL

```
itdns.canterbury.ac.nz    A
  132.181.2.225
```

Query & response: an example

QUERY client → local DNS · UDP 53

```
Transaction ID  0x6492
Flags           0x0100 (query, RD=1)
Questions  1    Answer RRs  0
Authority RRs  0    Additional RRs  0
```

QUESTIONS

```
www.canterbury.ac.nz    A
```

Then: an A record

The canonical name's IP address, 132.181.106.9 — the answer we wanted.

RESPONSE local DNS → client

```
Transaction ID  0x6492
Flags           0x8180 (reply, RA=1, no error)
Questions  1    Answer RRs  2
Authority RRs  1    Additional RRs  1
```

QUESTIONS

```
www.canterbury.ac.nz    A
```

ANSWERS

```
www.canterbury.ac.nz    CNAME
  web.canterbury.ac.nz
web.canterbury.ac.nz    A
  132.181.106.9
```

AUTHORITY

```
canterbury.ac.nz    NS
  itdns.canterbury.ac.nz
```

ADDITIONAL

```
itdns.canterbury.ac.nz    A
  132.181.2.225
```

Query & response: an example

QUERY client → local DNS · UDP 53

```
Transaction ID  0x6492
Flags           0x0100 (query, RD=1)
Questions  1    Answer RRs  0
Authority RRs  0    Additional RRs  0
```

QUESTIONS

www.canterbury.ac.nz **A**

Authority

An **NS** record naming an authoritative server for `canterbury.ac.nz`: the final level, which holds the answer's record.

RESPONSE local DNS → client

```
Transaction ID  0x6492
Flags           0x8180 (reply, RA=1, no error)
Questions  1    Answer RRs  2
Authority RRs  1    Additional RRs  1
```

QUESTIONS

www.canterbury.ac.nz **A**

ANSWERS

www.canterbury.ac.nz CNAME
  web.canterbury.ac.nz

web.canterbury.ac.nz A
  132.181.106.9

AUTHORITY

canterbury.ac.nz NS
  itdns.canterbury.ac.nz

ADDITIONAL

itdns.canterbury.ac.nz A
  132.181.2.225

Query & response: an example

QUERY client → local DNS · UDP 53

```
Transaction ID  0x6492
Flags           0x0100 (query, RD=1)
Questions  1    Answer RRs  0
Authority RRs  0    Additional RRs  0
```

QUESTIONS

```
www.canterbury.ac.nz    A
```

Additional

The **IP address** of that name server, so it can be reached directly.

RESPONSE local DNS → client

```
Transaction ID  0x6492
Flags           0x8180 (reply, RA=1, no error)
Questions  1    Answer RRs  2
Authority RRs  1    Additional RRs  1
```

QUESTIONS

```
www.canterbury.ac.nz    A
```

ANSWERS

```
www.canterbury.ac.nz    CNAME
  web.canterbury.ac.nz
web.canterbury.ac.nz    A
  132.181.106.9
```

AUTHORITY

```
canterbury.ac.nz    NS
  itdns.canterbury.ac.nz
```

ADDITIONAL

```
itdns.canterbury.ac.nz    A
  132.181.2.225
```

DNS

DNS — key takeaways

- **Main purpose** — DNS maps human-readable hostnames to IP addresses.
- **Hierarchy** — DNS is distributed: no single server holds all records.
  Responsibility is delegated across Root, TLD, and Authoritative servers.
- **Traversal pattern** — the client queries the local DNS resolver, which then follows referrals Root → TLD → Authoritative.
  Results are cached locally by TTL to avoid repeated lookups.

References

James F. Kurose & Keith W. Ross, *Computer Networking: A Top-Down Approach*, Addison Wesley, 3rd edition, 2005.

Chapter 2 Application Layer (2.1, 2.2, 2.4, 2.5)

William Stallings, *Data and Computer Communications*, Pearson, 10th edition, 2013.

Chapter 24 Electronic Mail, DNS, and HTTP (24.1–24.3)

Andrew S. Tanenbaum & David J. Wetherall, *Computer Networks*, Prentice Hall, 5th edition, 2010.

Chapter 7 The Application Layer (7.1–7.3)

Acknowledgements

These lecture notes are developed based on slides and notes from:

Barry Wu's lecture notes for COSC264, University of Canterbury

Dr Mengmeng Ge's lecture notes for COSC264, University of Canterbury, 2024

Assoc Prof Dan Kim's lecture notes for COSC264, University of Canterbury, 2017

Prof Aleksandar Kuzmanovic's lecture notes for CS340, Northwestern University, 2005